

A New Topic

We've now finished the section of the course dealing with computability, so let's move on to the topic of complexity.

That is, if a problem is decidable, how hard is it to solve — how much time or memory will we need to solve it?

A Motivating Example

Let's look at two somewhat similar problems:

- the Hamiltonian Path problem (HAM-PATH), and
- the Minimal Spanning Tree problem MST.

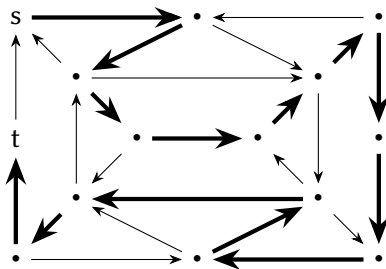
The Hamiltonian Path Problem

Definition: HAM-PATH

Instance: A directed graph G with specified vertices s and t .

Question: Does there exist a Hamiltonian path in G from s to t ?
(i.e., a path from s to t that passes through each vertex exactly once)

Example:



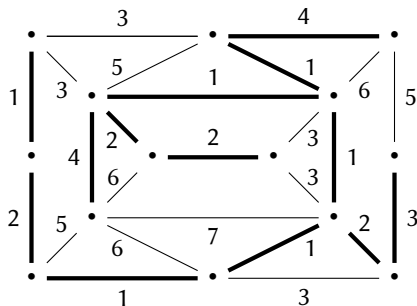
The Minimal Spanning Tree Problem

Definition: MST

Instance: A graph G with weights on its edges, a positive number k .

Question: Does G have a minimum spanning tree with weight at most k ?

Example:



How difficult are these programs to solve?

Are these problems decidable?

Yes — if a graph G has m edges and n vertices, a brute-force search for HAM-PATH will check $(n - 2)!$ paths. A brute-force search for MST will look at the $\mathcal{O}(n^{n-2})$ possible spanning trees and pick the smallest.

Note:

We'll be using $\mathcal{O}$ notation in this section of the course. Make sure you review this if you're not already familiar with it.

So our problem is solved — these problems are decidable! Once we write a program that solves them we're done, right?

... But we'd probably like to do a bit better than $\mathcal{O}(n!)$ or $\mathcal{O}(n^{n-2})$ time.

Assumptions on Computational Time

This section of the course assumes the following to take $\mathcal{O}(1)$ time each (this is similar to CSCB63):

- pointer comparison
- read/write 64 bits (data or pointer), if the address is also 64-bit
- branching, jumping

This isn't entirely accurate, but it doesn't change whether a program is polytime or not, and it makes our analysis easier. So we'll use this as a simplification until we need to be more accurate.

This course does not assume that integer arithmetic is $\mathcal{O}(1)$ time.

Can We do Better?

- The MST problem: solved by, e.g., Kruskal's algorithm ($\mathcal{O}(mn) = \mathcal{O}(n^3)$ time).

We can even do as well as $\mathcal{O}(n^2 \log(n))$ time.

- The HAM-PATH problem: the best we've done is about $\mathcal{O}(n^2 2^n)$.

We think this is about the best we can do.

Question: *Why do we get such a large difference between these two problems?*

Question: *Can we ever hope to build an algorithm for HAM-PATH that is even close to the efficiency that we can achieve for the MST problem?*

Conjecture: We believe that there is no such algorithm — that there is something fundamental about HAM-PATH such that we simply cannot solve it efficiently.

This is a conjecture, not a theorem: we do not yet know if the statement is true, but it is widely believed to be so.

Computational Complexity

In order to even start answering the questions posed above — that is, what are the most efficient ways of solving these problems, and why do they have those efficiencies — we'll have to define exactly what we mean when we say a problem is efficiently solvable.

We'll approach this by looking at the programs that solve these problems.

Question: *How can we measure the complexity of a program?*

There are a number of ways of doing this, but in this course we'll mostly look at two:

- **Time complexity:** How many steps does a program take in its computation?
- **Space complexity:** How much memory does a program use in its computation?

We'll spend the next several lectures looking at time complexity.

The Classes $\text{TIME}(t(n))$

Recall that a TM M has a *worst-case time* of $\mathcal{O}(f(n))$ if $t_M(n) = \mathcal{O}(f(n))$, where $t_M(n)$ is the longest running time M can take for any input of size n .

But we're not interested in a single program — remember that we're asking whether a *problem* can be computed efficiently.

When we talked about computability, we started by looking at yes/no questions (and their associated languages). We'll do the same thing here. We'll move to other problems later.

Definition: Let's define the class $\text{TIME}(t(n))$, for any function $t : \mathbb{N} \rightarrow \mathbb{R}_+$ to be

$$\text{TIME}(t(n)) = \left\{ L \mid \begin{array}{l} L \text{ is a language decided by some TM} \\ \text{that has a worst-case time of } \mathcal{O}(t(n)) \end{array} \right\}$$

Now we can argue that functions that grow relatively slowly, such as linear functions, are efficient. So the class $\text{TIME}(n)$, or even $\text{TIME}(n^2)$ would seem to be a reasonable description of the problems that can be efficiently solved.

But...

We should make sure our definition of efficiently solvable programs is robust — it should not vary depending on which TM formalism we use.

Question: *Does our TM model matter when determining which class a problem lies in?*

It turns out that it does: Consider the language $L = \{0^n 1^n \mid n \in \mathbb{N}\}$.

Best Possible Algorithms:

- For a single-tape TM: $\mathcal{O}(n \log(n))$ time.

We can actually prove this: Look at question 7.49 in the text.

- For a multitape TM: $\mathcal{O}(n)$ time.

We get a difference in time, but not by more than a power of two.

In general: When we are asking about computation times, the choice of TMs formalism matters, but only up to a constant power (as long as the TM can be feasibly implemented).

The Class $\mathcal{P}$

So when we ask which problems can be solved efficiently:

- Don't ask if they can be solved in, e.g., linear time (depends on formalism).
- Ask if they can be solved in some constant power of linear time: i.e., in polynomial time

Definition: We say the class $\mathcal{P}$ is the set of languages that can be decided in polynomial time on a deterministic single tape TM:

$$\mathcal{P} = \bigcup_{k \in \mathbb{N}} \text{TIME}(n^k).$$

This gives us a class of languages that can be decided efficiently, and which is robust across different TM formalisms.

Regarding Input Sizes

When we looked at the complexity of the algorithms for HAM-PATH and MST, we used the parameters m and n , where m was the number of edges and n was the number of vertices.

Question: *If we define the time complexity of our algorithms based on $|I|$, where I is the input, how does this change the complexity of our algorithms?*

Remember that we have to list both n and m in our input. So $n \leq |I|$ and $m \leq |I|$. This means we get:

$$n^2 2^n \leq |I|^2 2^{|I|}$$

and

$$m \log(n) \leq |I| \log |I|$$

So we can use these “natural inputs” to our problem without changing the whether the resulting problem is in $\mathcal{P}$.

Warning: *What if we write a number in binary or decimal notation?*

Complexity and Nondeterministic TMs

When we introduced the class $\mathcal{P}$, we said this is robust across the feasible TM formalisms.

But what about NTMs?

There's an exponential slowdown, but then, nondeterministic TMs are not possible to build.

But, many real-world problems are more naturally expressed in terms of nondeterminism than in terms of determinism.

E.g., HAM-PATH can be decided by an efficient NTM, even if we don't believe it is in $\mathcal{P}$.

Question: So how do we measure time complexity for NTMs?

Suppose that N is an NTM that decides some language L .

The running time $f(n)$ of N is the maximum number of steps taken on *any branch* of the computation tree that N takes for any input of size n .

The Classes $\text{NTIME}(t(n))$ and $\mathcal{NP}$

Definition: We can define the class $\text{NTIME}(t(n))$ using the same approach we used in the deterministic setting: For any function $t : \mathbb{N} \rightarrow \mathbb{R}_+$, we say that

$$\text{NTIME}(t(n)) = \left\{ L \mid \begin{array}{l} L \text{ is a language decided by some nondeterministic TM} \\ \text{that has a running time time of } \mathcal{O}(t(n)). \end{array} \right\}$$

As we discussed in the last class, the problems that can be solved in polynomial time are the ones that we consider to be efficient. We can make an analogous definition for NTMs.

Definition: We say the class $\mathcal{NP}$ is the set of languages that can be decided in polynomial time on a nondeterministic TM:

$$\mathcal{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k).$$

$\mathcal{P}$ versus $\mathcal{NP}$

Question: *What is the relationship between $\mathcal{P}$ and $\mathcal{NP}$?*

Answer: We don't actually know.

We do have a few details:

- Since every TM is also an NTM, every language in $\mathcal{P}$ is also in $\mathcal{NP}$:

$$\mathcal{P} \subseteq \mathcal{NP}.$$

- Since every NTM can be simulated by a TM with an exponential slowdown, every language in $\mathcal{NP}$ can be solved in $2^{n^{\mathcal{O}(1)}}$ time:

Any $L \in \mathcal{NP}$ can be decided in exponential time.

But we believe that they are different — that $\mathcal{P}$ is a proper subset of $\mathcal{NP}$.

Conjecture: $\mathcal{P} \neq \mathcal{NP}$

This is one of the most important open problems in Computer Science.

How does this relate to what we've already done?

Let's have a look again at HAM-PATH.

- We don't expect that HAM-PATH can be solved efficiently: Suppose I give you a graph G and a pair s and t of nodes, and then tell you that G has a Hamiltonian path from s to t . You can't check this very easily, and might well decide not to believe me.

But HAM-PATH is in $\mathcal{NP}$...

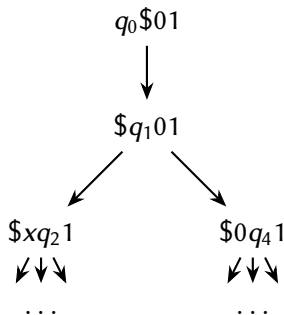
- **High-level non-deterministic algorithm for $\mathcal{NP}$:**
 - Non-deterministically choose a path P (a sequence of vertices).
 - Check whether P is a hamiltonian path.
 - If it is, *accept*, otherwise *reject*.

Alternatively, I can tell you what the Hamiltonian path P in G actually is. You can check whether I've given you a Hamiltonian path very easily.

Is this starting to sound a bit like the difference between decidability and recognizability?

$\mathcal{NP}$ and Certificates

Remember that the computation history of an NTM follows a tree structure:



Question: *What if, instead of extending every path, I told you beforehand which path to search?*

$\mathcal{NP}$ and Certificates (2)

This doesn't change the complexity of the computation — we still have to actually find an accepting computation, but we're looking for that path by choosing which branches to search before running the NTM.

Searching every path is a bit like doing a DFS on the tree, which is allowable since we know the tree has finite depth.

Overall:

- We can search a single path of the computation tree of a nondeterministic TM by choosing that path to start off, then checking that path on the tree.
- Following a single path of the tree is a deterministic process.
- If I tell you that a nondeterministic TM N accepts some input x , I'm really saying that there's some branch of its computation tree rooted at x that accepts.

$\mathcal{NP}$ and Certificates (3)

If $N \in \mathcal{NP}$, then this branch has a length that's polynomial in $|x|$. So I can tell you which branch to take with a description that's polynomial in $|x|$.

This means we can describe $\mathcal{NP}$ as follows:

An Alternative Description of $\mathcal{NP}$:

A language $L \in \mathcal{NP}$ if there is a deterministic polytime TM V such that $x \in L$ iff $\exists$ a string C_x that is polynomial in the length of x such that $V(x, C_x)$ accepts.

The TM V is a verifier, and the string C_x is a certificate.

Showing a Language is in $\mathcal{NP}$

To show a language L is in $\mathcal{NP}$, we follow five steps:

1. Show that L is a decision problem.
2. Give a certificate for L .
3. Show the certificate is polynomial in the input size.
4. Give a verifier for the certificate.
5. Show the verifier is polytime in the input size.

Example 1: HAM-PATH

Input: A graph G with specified vertices s and t .

Question: Does there exist a Hamiltonian path in G from s to t ?

Certificate: An ordered list $v_1, v_2, \dots, v_k$ of nodes in G

Is this polynomial in the size of G ?

Verifier: On input $\langle G, s, t, C \rangle$:

1. Check that $v_1 = s$.
2. Check that $v_k = t$.
3. Check that $k = |V|$.
4. For $i = 1$ to $k - 1$:
5. Check that there is an edge from v_i to v_{i+1} .
6. Check that every vertex v in G occurs exactly once in the list.
7. If every one of these checks passes, *accept*. Otherwise *reject*.

Is this polynomial in the size of G ?

Example 2: CLIQUE

Input: A graph G and a number $k \in \mathbb{N}$.

Question: Does G have a clique of size k ?

A clique S is a collection of vertices in G such that for any $u, v \in S$, there is an edge between u and v .

Certificate: A set $v_1, v_2, \dots, v_r$ of nodes in G .

Is this polynomial in the size of G ?

Verifier: On input $\langle G, k, C \rangle$:

1. Check that $r \geq k$.
2. For every pair u, v of vertices in C :
3. Check that there is an edge from u to v .
4. If every one of these checks passes, *accept*. Otherwise *reject*.

Is this polynomial in the size of G ?

Example 3: PATH

Input: A graph G with specified vertices s and t .

Question: Does there exist a path in G from s to t ?

Notice that this time we're just asking for a path, not a Hamiltonian path – it doesn't have to pass through every vertex.

Certificate: an ordered list $v_1, v_2, \dots, v_k$ of nodes in G

Is this polynomial in the size of G ?

Verifier: On input $\langle G, s, t, C \rangle$:

1. Check that $v_1 = s$.
2. Check that $v_k = t$.
3. For $i = 1$ to $k - 1$:
4. Check that there is an edge from v_i to v_{i+1} .
5. If every one of these checks passes, *accept*. Otherwise *reject*.

Is this polynomial in the size of G ?

Example 4: 3SAT

Input: A boolean formula ϕ in 3CNF.

3CNF just means that ϕ looks something like:

$$(x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_4} \vee x_3 \vee \overline{x_1}) \wedge \dots \wedge (x_k \vee \overline{x_{k-1}} \vee x_2).$$

It's a conjunction of several clauses, each of which is a disjunction of exactly three boolean variables

Question: Does ϕ have a satisfying assignment?

Certificate:

A truth assignment $\tau : x_1 = T/F, x_2 = T/F, \dots, x_k = T/F$

Is this polynomial in the size of ϕ ?

Verifier: On input $\langle \phi, \tau \rangle$:

1. Set every variable in ϕ according to the truth assignment τ and evaluate the resulting expression.
2. If ϕ evaluates to TRUE with the truth assignment τ , *accept*. Otherwise, *reject*.

Is this polynomial in the size of G ?

Decidability and Polytime Decidability

Recall:

$\mathcal{P}$ is the class of problems that can be solved in polynomial time.

i.e., Problems that can be solved efficiently.

$\mathcal{NP}$ is the class of problems that can be verified in polynomial time.

i.e., Problems whose solutions can be checked efficiently.

There is a parallel between these ideas and the ideas of decidability and recognizability:

The **decidable** languages can be solved in a finite amount of time.

i.e., Problems that can be solved at all.

The **recognizable** languages can be verified in any amount of time.

i.e., Problems whose solutions can be checked at all.

How Far Can We Push This Analogy?

Here are a few of the ideas from the first half of the course:

Computability	Complexity
The decidable languages	The class $\mathcal{P}$
The recognizable languages	The class $\mathcal{NP}$
The co-recognizable languages	????
Mapping reductions	????
The language HALT	????

Question: *Do these ideas have parallels when we're looking at efficient computation instead of general computation?*

The Class $\text{co-}\mathcal{NP}$

The first parallel we'll give is to extend the idea of co-recognizability — we'll describe the class $\text{co-}\mathcal{NP}$:

Definition: The class $\text{co-}\mathcal{NP}$ is the set of languages L whose complements are in $\mathcal{NP}$:

$$L \in \text{co-}\mathcal{NP} \iff \bar{L} \in \mathcal{NP}.$$

We have conjectured (again) that this class is separate from $\mathcal{NP}$.

We can show a language L is in $\text{co-}\mathcal{NP}$ by using the five steps from earlier to show $\bar{L} \in \mathcal{NP}$:

1. Show that L (and so $\bar{L}$) is a yes/no question.
2. Give a certificate for $\bar{L}$.
3. Show this certificate is polynomial in the size of the input to L .
4. Give a verifier for $\bar{L}$.
5. Show this verifier is polytime in the size of the input to L .

How are the classes $\mathcal{P}$, $\mathcal{NP}$, and $\text{co-}\mathcal{NP}$ related?

We don't know: In fact, the following questions are still unresolved, and are among the "Millennium Problems" recognized by the Clay Mathematics Institute. If you can resolve any of them you can win a million dollars:

- Is $\mathcal{P} = \mathcal{NP} \cap \text{co-}\mathcal{NP}$?
(no strong consensus)
- Is $\mathcal{NP} = \text{co-}\mathcal{NP}$?
(strongly believed to be no)
- Is $\mathcal{P} = \mathcal{NP}$?
(strongly believed to be no)

Polytime Reductions

The second parallel we'll give is to the concept of mapping reductions.

Recall: $A \leq_m B$ iff there is a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that

$$\forall w \in \Sigma^*, w \in A \leftrightarrow f(w) \in B.$$

The Big Idea: If we could determine whether $w \in B$, we could also determine whether $x \in A$ by asking whether $f(x) \in B$.

We can do the same thing in the polytime realm by simply requiring that f be polytime:

Definition: We say A is polytime reducible to B ($A \leq_p B$) iff there is a polytime computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that

$$\forall w \in \Sigma^*, w \in A \leftrightarrow f(w) \in B.$$

What This Means:

Theorem 7.31:

If $A \leq_p B$ and $B \in \mathcal{P}$, then so is A .

If $A \leq_p B$ and $B \in \mathcal{NP}$, then so is A .

Corollary:

If $A \leq_p B$ and $A \notin \mathcal{P}$, then neither is B .

If $A \leq_p B$ and $A \notin \mathcal{NP}$, then neither is B .

This is similar to our use of mapping reductions:

If $A \leq_m B$ and A is not decidable, then neither is B .

If $A \leq_p B$ and A cannot be solved in polytime, then neither can B .

An Analogue to HALT

Problem:

- When we started showing languages were undecidable, we started by showing the languages A_{TM} and $HALT$ were undecidable. We went on from there by making use of the fact that these languages are undecidable.
- But it's going to be difficult to start with a language that we know is in $\mathcal{NP} \setminus \mathcal{P}$, since we haven't proven that these classes are unequal.
- *So what should we be reducing from?*

Question: *Why is $HALT$ a good language to use in our reductions?*

$HALT$ can be thought of as one of the hardest recognizable languages:

For any recognizable language L , $L \leq_m HALT$.

Hardness and Completeness

Why? You've already seen this idea in tutorial 4: If L is recognizable, then it has an recognizer R . So we can write the TM

Let $R' =$ “ On input $\langle x \rangle$:

1. Run R on $\langle x \rangle$.
2. If it accepts, *accept*, otherwise *loop*.

We can clearly see that $w \in L$ iff $\langle R', w \rangle \in \text{HALT}$.

Definition: For any class $\mathcal{C}$ of languages, we say that a language L is *hard* for that class if every $A \in \mathcal{C}$ reduces to it.

- So HALT is *hard for the recognizable languages*.

Definition: For any class $\mathcal{C}$ of languages, we say that a language L is *complete* for that class if $L \in \mathcal{C}$ and every $A \in \mathcal{C}$ reduces to it.

- So HALT is *complete for the recognizable languages*.

Is there an $\mathcal{NP}$ -complete language?

Answer: Yes, there is.

Cook-Levin Theorem: The language 3SAT is $\mathcal{NP}$ -complete.

Note that when we were talking about hardness and completeness for recognizable languages, reducible meant mapping reducible. Now we mean polytime reducible.

What this means: if $3\text{SAT} \in \mathcal{P}$, then any problem in $\mathcal{NP}$ can also be solved in polynomial time.

Since we believe this is not the case, we strongly suspect that $3\text{SAT} \notin \mathcal{P}$

Furthermore, if $L \in \mathcal{NP}$ and $3\text{SAT} \leq_p L$, then L is also $\mathcal{NP}$ -complete.

If $\mathcal{P} \neq \mathcal{NP}$, then no such L is in $\mathcal{P}$.

A Few More Examples: 3COL (3-Colourability)

INPUT: A set $G = (V, E)$.

QUESTION: Is there a way of colouring the vertices with three colours in G so that whenever $\{u, v\} \in E$, u and v have different colours?

Certificate: The colouring C

Is this polynomial in the size of ϕ ?

Let $\mathcal{V}$ = “On $\langle G = (V, E), C \rangle$:”

1. Check that C maps every vertex to one of three colours (we'll refer to them as *red*, *green*, and *blue*).
2. For all edges $\{u, v\} \in E$:
3. Check that $C[u] \neq C[v]$.
4. If all of these checks pass, *accept*. Otherwise, *reject*.

Is this polynomial in the size of G ?

A Few More Examples: X1-3SAT (Exactly-One-3SAT)

INPUT: A 3CNF ϕ .

QUESTION: Does ϕ have a satisfying assignment in which exactly one literal in each clause is set to true?

Certificate:

A truth assignment $\tau : x_1 = T/F, x_2 = T/F, \dots, x_k = T/F$

Is this polynomial in the size of ϕ ?

Verifier: On input $\langle \phi, \tau \rangle$:

1. Set every variable in ϕ according to the truth assignment τ and evaluate the resulting expression.
2. If ϕ sets exactly one literal per clause to TRUE with the truth assignment τ , *accept*. Otherwise, *reject*.

Is this polynomial in the size of G ?

A Few More Examples: NAE-3SAT (Not-All-Equal-3SAT)

INPUT: A 3CNF ϕ .

QUESTION: Does ϕ have a satisfying assignment in which no clause has all literals set to false or to true?

Certificate:

A truth assignment $\tau : x_1 = T/F, x_2 = T/F, \dots, x_k = T/F$

Is this polynomial in the size of ϕ ?

Verifier: On input $\langle \phi, \tau \rangle$:

1. Set every variable in ϕ according to the truth assignment τ and evaluate the resulting expression.
2. If ϕ sets exactly one or two literals per clause to TRUE with the truth assignment τ , *accept*. Otherwise, *reject*.

Is this polynomial in the size of G ?

A Few More Examples: 3DM (3-Direct-Matching)

INPUT: Three sets A, B, C , all of size n , and a set T of triples in $A \times B \times C$.

QUESTION: Is there an $M \subseteq T$ such that every element of A, B , and C occurs exactly once in M ?

Certificate: The subset M

Is this polynomial in the size of ϕ ?

Let $\mathcal{V} = \text{"On } \langle A, B, C, T, M \rangle \text{"}$

1. Check that $M \subseteq T$.
2. Check that $|M| = |A|$.
3. Check that for any distinct (a, b, c)
and $(a', b', c') \in M$, $a \neq a'$, $b \neq b'$, and $c \neq c'$.
4. If all of these checks pass, *accept*, otherwise *reject*.

Is this polynomial in the size of G ?